ReAct vs Plan-then-Execute, Costed by Hand

Two control architectures, one token meter — and the exact step count where "plan first" stops paying and starts saving.

What this document does. Module 0.1 sets ReAct ("Thought → Action → Observe → …") against Plan-then-Execute ("Thought → [Plan steps] → Execute all") and labels the second "cheap (1 planning call)" — but never says *how* cheap, or *when* the swap is worth it. Here we cost both, token by token, for the same Customer-Support Resolver, using the identical constants from Module 0.0 Topic 1. We rebuild $C_{\text{react}}(N)$, derive $C_{\text{plan}}(k)$ from scratch, and find the break-even step count. Every number is reproduced by the Python program in Appendix A.

Where this sits. This is **Topic 2 of Module 0.1**. Its companions: Module 0.0 Topic 1 (the ReAct cost curve we reuse verbatim here) and Module 0.1 Topic 1 (context-window pressure — where Plan-then-Execute's flat transcript also dodges the overflow wall). The lever this document isolates is the one the cost document named: *stop re-feeding history.*

Concepts covered, in order: the ReAct cost $C_{\text{react}}(N) = p_{\text{in}}[NB + h\frac{N(N-1)}{2}] + p_{\text{out}}Na$ · why it is $O(N^2)$ · Plan-then-Execute as one planning call + k history-free executions · deriving $C_{\text{plan}}(k)$ as *linear* in k · the break-even step · ReAct for short adaptive tasks, Plan for long stable ones.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: two architectures, one price list

Both architectures solve the same task and use the same per-message sizes; they differ only in *what they re-send each call.* That single difference is the whole result.

Quantity	Symbol	Value here
Base context, re-sent every call	B	2,000 tokens
History added per completed step (ReAct)	h	1,800 tokens
Assistant / answer message (output)	a	300 tokens
Plan output per planned step	plan	200 tokens
Step's own result fed to an exec call	r	200 tokens
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens
Steps in the task	$N = k$	$3, 5, 8, 12$

Notation. For a fair race we set the number of executable steps equal: a k-step plan is compared to an $N = k$-step ReAct run. $C_{\text{react}}(N)$ and $C_{\text{plan}}(k)$ are total dollars. Prices and B, h, a are identical to Module 0.0 Topic 1, so the ReAct column here matches that document line for line.

Intuition

ReAct re-reads the whole case file before every action — so the later actions drag a heavy history. Plan-then-Execute reads the file *once*, writes down the full to-do list, then works each item with only the list and that item in hand: it never re-reads the growing pile. The question is purely: does the up-front planning call cost less than the history ReAct accumulates? For a two-step errand, no. For a twelve-step march, overwhelmingly yes.

2 Step 1 — the ReAct cost, recalled

From Module 0.0 Topic 1, summing the growing prompt $P(t) = B + (t-1)h$ over N steps gives total input $NB + h\frac{N(N-1)}{2}$, and output Na. So

$$\boxed{C_{\text{react}}(N) \;=\; p_{\text{in}}\left[NB + h\,\tfrac{N(N-1)}{2}\right] \;+\; p_{\text{out}}\,Na} \tag{1}$$

The $h\frac{N(N-1)}{2}$ term is the triangular number — the quadratic history cost. It is the entire reason a long ReAct run gets expensive, and the entire thing Plan-then-Execute is built to avoid.

Mini-example — this operation in isolation

$N = 3$: input $= 3 \cdot 2000 + 1800 \cdot \frac{3 \cdot 2}{2} = 6000 + 5400 = 11{,}400$ tokens; output $= 3 \cdot 300 = 900$. Cost $= 11400 \cdot 3 \times 10^{-6} + 900 \cdot 15 \times 10^{-6} = 0.0342 + 0.0135 = \$0.0477$. ✓ Identical to the three-step trajectory in Module 0.0.

3 Step 2 — deriving the Plan-then-Execute cost

Plan-then-Execute is two phases. Count the tokens of each.

Phase 1 — one planning call. The planner reads the base context B (system + tools + goal) and emits a k-step plan. We size the plan output at plan $= 200$ tokens per step, so its output is $k \cdot \text{plan}$. Planning input/output:

$$\text{in}_{\text{plan}} = B, \qquad \text{out}_{\text{plan}} = k \cdot \text{plan}.$$

Phase 2 — k execution calls, history-free. This is the crux. Because the plan is *fixed*, each execution call does *not* re-feed the accumulating transcript. It carries only the base B (it still needs the system prompt, the relevant tool, and the goal) plus its own step's small result r, and emits an answer message a. So per exec call: input $B + r$, output a. Over k calls:

$$\text{in}_{\text{exec}} = k\,(B + r), \qquad \text{out}_{\text{exec}} = k\,a.$$

Add the phases and attach prices:

$$\boxed{C_{\text{plan}}(k) \;=\; p_{\text{in}}\big[\,B + k(B + r)\,\big] \;+\; p_{\text{out}}\big[\,k \cdot \text{plan} + k\,a\,\big]} \tag{2}$$

The decisive feature: every term is *linear* in k. There is no $\frac{k(k-1)}{2}$. The quadratic history term is simply absent, because no call ever re-reads another call's output. That is the whole saving.

Mini-example — this operation in isolation

$k = 3$: input $= B + 3(B + r) = 2000 + 3 \cdot 2200 = 2000 + 6600 = 8{,}600$ tokens; output $= 3 \cdot 200 + 3 \cdot 300 = 600 + 900 = 1{,}500$. Cost $= 8600 \cdot 3 \times 10^{-6} + 1500 \cdot 15 \times 10^{-6} = 0.0258 + 0.0225 = \$0.0483$. ✓ Compare ReAct's \$0.0477 — at three steps they are a dead heat (ReAct a hair cheaper).

Watch out

The "history NOT re-fed" assumption is what makes Plan linear — and it is also Plan's weakness. If a mid-plan step needs the result of an earlier step, you must either widen r (feed more back, eating the saving) or accept that the plan was written blind. This is the module's "brittle to deviations / plan can be wrong" caveat, in cost form: the saving is real only when the steps are genuinely independent of each other's *observations*.

4 Step 3 — linear vs quadratic, side by side

Tabulate both at $N = k \in \{3, 5, 8, 12\}$ with the constants above.

$N{=}k$	C_{react} in-tok	C_{react}	C_{plan} in-tok	C_{plan}	**cheaper**
3	11,400	\$0.0477	8,600	\$0.0483	ReAct (barely)
5	28,000	\$0.1065	13,000	\$0.0765	**Plan**
8	66,400	\$0.2352	19,600	\$0.1188	**Plan**
12	142,800	\$0.4824	28,400	\$0.1752	**Plan**

Sample check, $k = 5$ plan: input $= 2000 + 5 \cdot 2200 = 13{,}000$; output $= 5 \cdot 200 + 5 \cdot 300 = 2{,}500$; cost $= 13000 \cdot 3 \times 10^{-6} + 2500 \cdot 15 \times 10^{-6} = 0.039 + 0.0375 = \$0.0765$. ✓ Meanwhile ReAct at $N = 5$ is \$0.1065 — Plan is now 1.4× cheaper, and the gap only widens: by $N = 12$, ReAct (\$0.4824) is 2.75× Plan (\$0.1752).

5 Step 4 — the break-even step

ReAct wins for tiny tasks (almost no history to carry, and it skips the planning call's overhead); Plan wins once history compounds. The crossover is where the two costs meet. From the verified figures:

$N{=}k$	C_{react}	C_{plan}	**verdict**
1	\$0.0105	\$0.0201	ReAct cheaper
2	\$0.0264	\$0.0342	ReAct cheaper
3	\$0.0477	\$0.0483	ReAct cheaper (margin \$0.0006)
4	\$0.0744	\$0.0624	**Plan cheaper ← crossover**
5	\$0.1065	\$0.0765	Plan cheaper

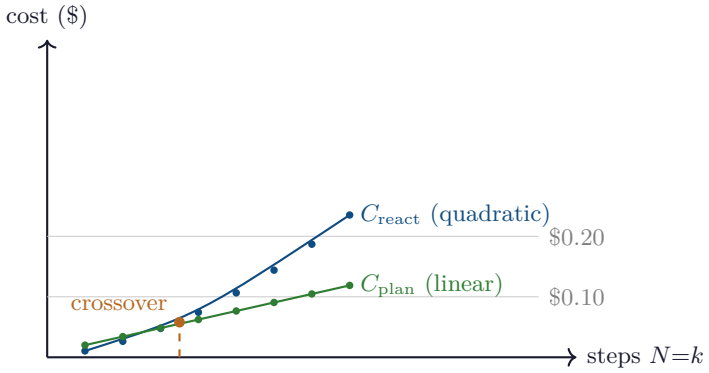
$$\boxed{\text{Break-even between } N = 3 \text{ and } N = 4: \quad \text{ReAct} \le \text{Plan for } N \le 3, \quad \text{Plan} < \text{ReAct for } N \ge 4.} \tag{3}$$

The mechanism is visible in the slopes. C_{plan} rises by a flat $\approx$ \$0.0141 per step (it is linear). C_{react}'s per-step increment *grows* — step 4 adds \$0.0267, step 5 adds \$0.0321 — because each new ReAct step re-reads one more slab of history than the last. A linear line and a steepening curve that start together must cross exactly once; here that is right after step 3.

Intuition

"Plan first" is not universally cheaper — it is cheaper *past the break-even*. For a 1–3 step errand, the planning call is pure overhead you never recoup, and ReAct's adaptivity is free. Past ~4 steps the quadratic history term overtakes that overhead and never looks back. So the architecture choice is really a length bet: short and surprising ⇒ ReAct; long and predictable ⇒ Plan. The 20–200-step research agents in Module 0.1 live deep in Plan territory — which is why long-horizon agents almost always plan, decompose, or checkpoint rather than run one ever-growing ReAct transcript.

6 The crossover, drawn



The blue ReAct curve starts *below* the green Plan line (no history, no planning overhead), but it bends upward — the triangular history term — while Plan stays straight. They cross just past step 3. Left of the dashed line ReAct is cheaper; right of it, and increasingly so, Plan wins. The picture is Equation (1) (curved) racing Equation (2) (straight).

7 Cost cheat-sheet

ReAct total cost	$C_{\text{react}}(N) = p_{\text{in}}\left[NB + h\frac{N(N-1)}{2}\right] + p_{\text{out}}Na$
ReAct growth	$O(N^2)$ — the history term $h\frac{N(N-1)}{2}$
Plan input tokens	$B + k(B + r)$
Plan output tokens	$k \cdot \text{plan} + k\,a$
Plan total cost	$C_{\text{plan}}(k) = p_{\text{in}}[B + k(B + r)] + p_{\text{out}}[k\,\text{plan} + ka]$
Plan growth	$O(k)$ — linear, no history term
Break-even (these constants)	between $N = 3$ and $N = 4$
Rule of thumb	short/adaptive ⇒ ReAct; long/stable ⇒ Plan

8 An honest word about these numbers

The break-even sits at "between 3 and 4 steps" *for these constants*. Move them and the crossover moves with them, predictably: a larger base B pushes break-even later (planning's fixed overhead matters more); a larger history slab h or a smaller fed-back result r pulls it earlier (ReAct's quadratic bites sooner, Plan stays lean). The plan-output size and the per-exec result r are the softest guesses here — a plan that must thread real intermediate results through each step (r large) erodes Plan's edge and can erase it entirely. What is *not* illustrative is the shape of the race: C_{react} is quadratic in steps and C_{plan} is linear, so for *some* step count Plan always wins, and that count is small whenever history is heavy. ReAct buys adaptivity at a quadratic price; Plan buys a linear price at the cost of adaptivity. Choosing between them is choosing which of those you can afford — the same lever (stop re-feeding history) that bends both the cost curve of Module 0.0 and the window curve of Topic 1.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce both cost formulas, the comparison table, and the break-even scan; change any constant to find your own crossover.

```python
# react_vs_plan.py -- reproduces every number in "ReAct vs Plan-then-Execute".
B = 2000 # base context (system + tools + goal), tokens
h = 1800 # ReAct history added per completed step
a = 300 # assistant / answer message (output), tokens
PLAN = 200 # plan output per planned step, tokens
r = 200 # step's own result fed into each exec call, tokens
P_IN = 3e-6 # input price, $ per token ($3 / 1M)
P_OUT = 15e-6 # output price, $ per token ($15 / 1M)

def c_react(N): # Eq. (1): quadratic in N
    t_in = N*B + h*(N*(N-1))//2
    t_out = N*a
    return t_in, t_out, t_in*P_IN + t_out*P_OUT

def c_plan(k): # Eq. (2): linear in k, no history term
    t_in = B + k*(B + r) # 1 planning read + k history-free exec reads
    t_out = k*PLAN + k*a # plan emission + k answers
    return t_in, t_out, t_in*P_IN + t_out*P_OUT
```

```python

# --- comparison table ---
print(f"{'N=k':>4} {'react_in':>9} {'react$':>9} {'plan_in':>8} {'plan$':>9} cheaper")
for n in (3, 5, 8, 12):
    ri, ro, rc = c_react(n)
    pi, po, pc = c_plan(n)
    print(f"{n:>4} {ri:>9} {rc:>9.4f} {pi:>8} {pc:>9.4f} "
          f"{'ReAct' if rc < pc else 'Plan'}")

# --- break-even scan ---
print("\nbreak-even scan:")
for n in range(1, 6):
    rc = c_react(n)[2]; pc = c_plan(n)[2]
    print(f" N={n}: react=${rc:.4f} plan=${pc:.4f} "
          f"-> {'ReAct' if rc < pc else 'Plan'} cheaper")

# Expected output:
# N=k react_in react$ plan_in plan$ cheaper
# 3 11400 0.0477 8600 0.0483 ReAct
# 5 28000 0.1065 13000 0.0765 Plan
# 8 66400 0.2352 19600 0.1188 Plan
# 12 142800 0.4824 28400 0.1752 Plan
#
# break-even scan:
# N=1: react=$0.0105 plan=$0.0201 -> ReAct cheaper
# N=2: react=$0.0264 plan=$0.0342 -> ReAct cheaper
# N=3: react=$0.0477 plan=$0.0483 -> ReAct cheaper
# N=4: react=$0.0744 plan=$0.0624 -> Plan cheaper
# N=5: react=$0.1065 plan=$0.0765 -> Plan cheaper
```

— end of document —